



Deterministic Annealing in CUDA - UPDATE

A. C. O. Santos¹, T. Tomei¹, A. Sugunan²

¹SPRACE, ²TIFR

Table of Contents

1. Introduction and Goals
2. Deterministic Annealing
3. CUDA GPU
4. CUDA Implementation
5. Conclusions

Introduction and Goals: High luminosity collateral effect - Pileup

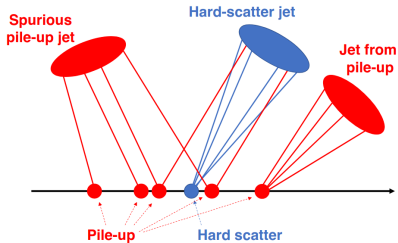
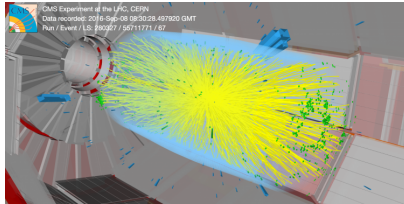


Figure: <https://cms.cern/>

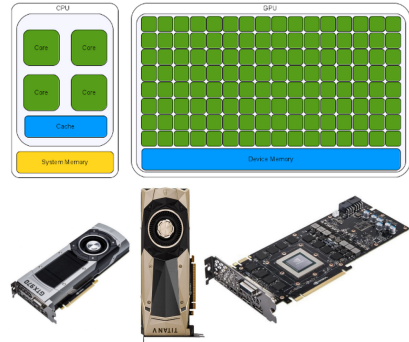


Figure: [10.1371/journal.pone.0097277](https://doi.org/10.1371/journal.pone.0097277)

Goal: Primary Vertex Position - Deterministic Annealing

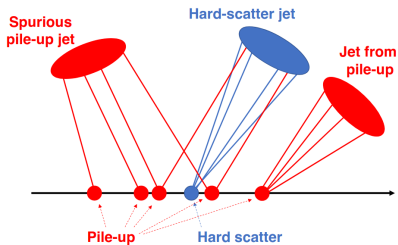
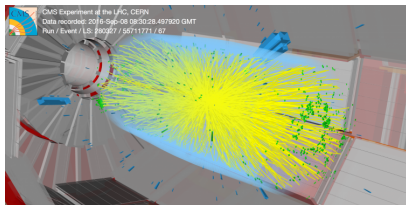


Figure: <https://cms.cern/>

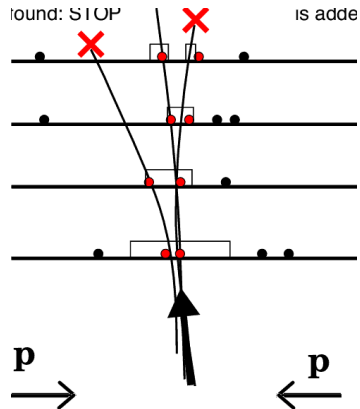


Figure: z-coordinate of their point of closest approach to the beam-line, [arXiv:0902.1860v2](https://arxiv.org/abs/0902.1860v2)

Deterministic Annealing

Consider index i related to **tracks** and k with **vertex** (always). Then we have the “Energy”

$$E_{ik} = \frac{(z_i - z_k)^2}{\sigma_i^2}, \quad E = \sum_k \sum_i p_i \rho_k p_{ik} E_{ik}. \quad (1)$$

Alternatively, we have the free energy

$$F_\beta(E_{ik}) = -\frac{1}{\beta} \sum_i \log \sum_k \exp(-\beta E_{ik}), \quad \beta = 1/T. \quad (2)$$

Constrained to

$$\left\{ \begin{array}{l} \sum_k \rho_k p_{ik} = 1, \\ \sum_k \rho_k = 1 \end{array} \right. . \quad (3)$$

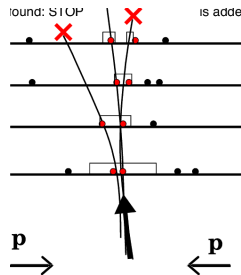


Figure: z-coordinate of their point of closest approach to the beam-line, [arXiv:0902.1860v2](https://arxiv.org/abs/0902.1860v2)

Deterministic Annealing

Consider index i related to **tracks** and k with **vertex** (always). Then we have the “Energy”

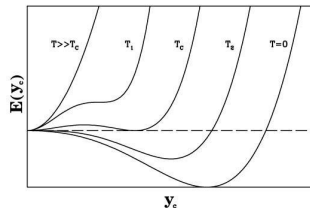
$$E_{ik} = \frac{(z_i - z_k)^2}{\sigma_i^2}, \quad E = \sum_k \sum_i p_i \rho_k p_{ik} E_{ik}. \quad (4)$$

Alternatively, we have the free energy

$$F_\beta(E_{ik}) = -\frac{1}{\beta} \sum_i \log \sum_k \exp(-\beta E_{ik}). \quad (5)$$

with $\beta = 1/T$. Constrained to

$$\begin{cases} \sum_k \rho_k p_{ik} = 1, \\ \sum_k \rho_k = 1 \end{cases}.$$



(6) Figure: Loss function to minimize
<https://ned.ipac.caltech.edu/>

Deterministic Annealing

In thermal equilibrium ($T = 1/\beta$) we have the probability update

$$p_{ik} = \frac{e^{-\beta E_{ik}}}{\sum_{k'} \rho_{k'} e^{-\beta E_{ik'}}}, \quad (7)$$

remember that $E_{ik} = (z_i - z_k)^2 / \sigma_i^2$, vertex position update

$$z_k = \frac{\sum_i p_i p_{ik} z_i / \sigma_i^2}{\sum_i p_i p_{ik} / \sigma_i^2}, \quad (8)$$

the weight

$$\rho_k = \frac{\rho_k \sum_i p_i p_{ik}}{\sum_i p_i}, \quad (9)$$

and the critical temperature*

$$T_c = 2 \frac{\sum_i \frac{p_i p_{ik}}{\sigma_i^2} \left(\frac{z_i - z_k}{\sigma_i} \right)^2}{\sum_i \frac{p_i p_{ik}}{\sigma_i^2}}. \quad (10)$$

Deterministic Annealing

At the beginning $\beta \approx 0$ ($T \rightarrow \infty$)

$$T_0 = 2 \frac{\sum_i \frac{p_i}{\sigma_i^2} \left(\frac{z_i - z_k}{\sigma_i} \right)^2}{\sum_i \frac{p_i}{\sigma_i^2}}, \quad (11)$$

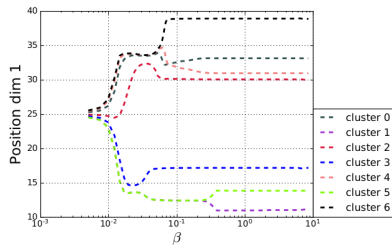
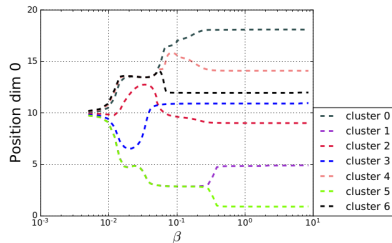
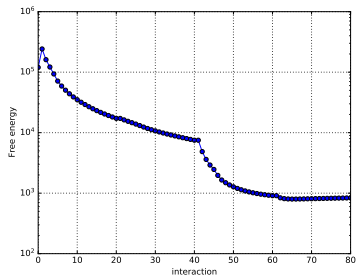
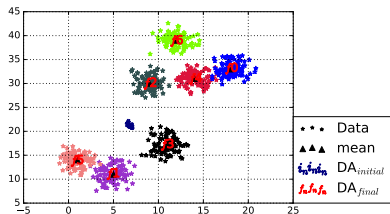
the vertex position is

$$z_0 = \frac{\sum_i p_i z_i / \sigma_i^2}{\sum_i p_i / \sigma_i^2}, \quad (12)$$

and weight (ρ_0) is one.

The annealing starts just above T_0 with two identical clusters at $z_0 \pm \delta$, with some small separation δ . As long as the Temperature is above T_c these two vertices will remain close together, but as soon as $T < T_c$ they separate.

Deterministic Annealing - Not CUDA yet (python)



GPU/CUDA Introduction and History*

□ Computer architectures

- SISD - Single instruction single data ;
- SIMD - Single instruction multiple data;
- MISD - Multiple instruction single data;
- MIMD - Multiple instruction multiple data;

□ SIMD - Single instruction multiple data.



- Graphic Processing Unit (GPU - 1994);
- GPGPU - General purpose GPU computing (2001);
- CUDA (2007) - Compute Unified Device Architecture (**Fortran**, **C/C++**, pyCUDA).

Cuda - Basic Ingredients

- Host, Device;
- Threads;
- Blocks;
- Grids;
- Cores;
- Streaming multiprocessors;
- Warps;
- Compile with `nvcc`;
- GPU programming is directly related to chip architecture!

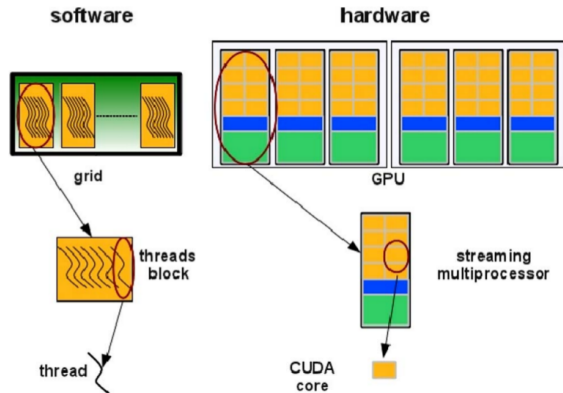


Figure:
[DOI:10.1371/journal.pone.0097277](https://doi.org/10.1371/journal.pone.0097277). Source:
PubMed

Cuda - Organization of threads in a CUDA program

□ threadIdx, blockIdx, blockDim, gridDim

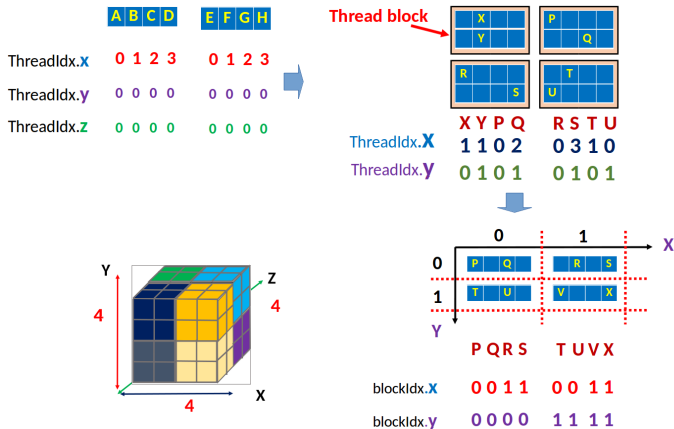


Figure: Cuda programming Masterclass. Kasun Liyanage.

Cuda - Organization of threads in a grid

- Unique index calculation for thread in a grid

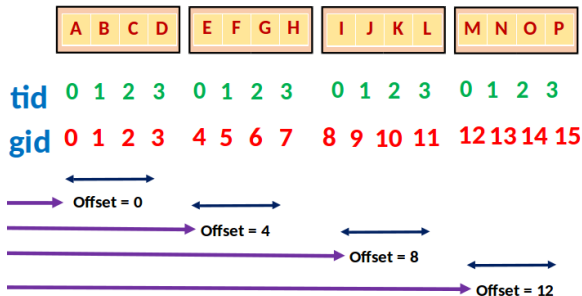


Figure: Cuda programming Masterclass. Kasun Liyanage.

- $gid = tid + \text{offset}$

- $gid = tid + \text{blockIdx.x} * \text{blockDim.x}$

- $gid = 3 = 3 + 0*4$

- $gid = 5 = 1 + 1*4$

- $gid = 14 = 2 + 3*4$

Cuda - Problems/Optimization - Memory Access

□ Bandwidth bound applications;

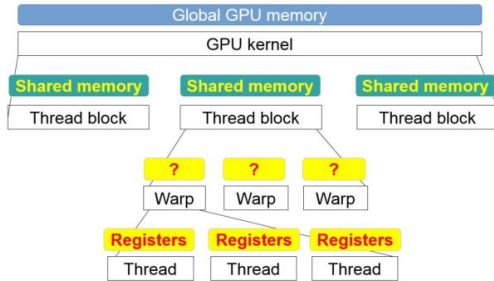


Figure: <https://developer.nvidia.com/blog/register-cache-warp-cuda/>

- Registers;
- Shared Memory;
- Constant memory;
- Texture Memory;
- Global Memory;
- GPU Caches.

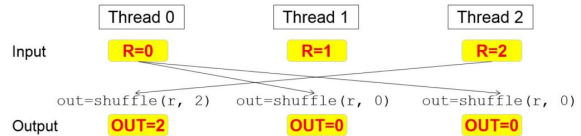
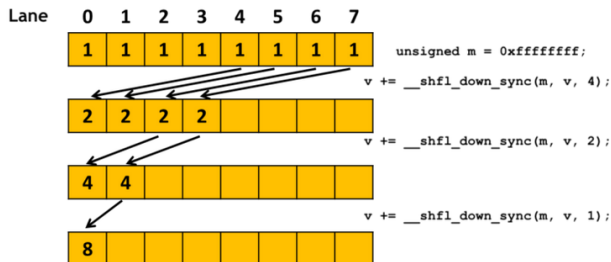


Figure: `shfl_sync()`

$$z_0 = \frac{\sum_i p_i z_i / \sigma_i^2}{\sum_i p_i / \sigma_i^2}, \quad T_0 = 2 \frac{\sum_i \frac{p_i}{\sigma_i^2} \left(\frac{z_i - z_k}{\sigma_i} \right)^2}{\sum_i \frac{p_i}{\sigma_i^2}} \quad (13)$$



Part of a warp-level parallel reduction using shfl_down_sync().

Figure: Ref: <https://developer.nvidia.com/blog/using-cuda-warp-level-primitives/>

Cuda - Problems/Optimization - Parallel Reduction

$$p_{ik} = \frac{e^{-\beta E_{ik}}}{\sum_{k'} \rho_{k'} e^{-\beta E_{ik'}}}, \quad E_{ik} = \frac{(z_i - z_k)^2}{\sigma_i^2},$$

`off = blockSize * offset + tid` $\Rightarrow$ `sum[gid] += sum[off];`

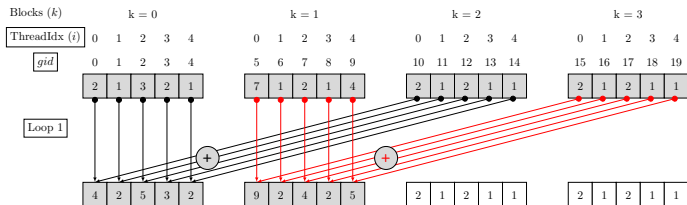
Blocks (k)						k = 0					k = 1					k = 2					k = 3				
ThreadIdx (i)						0	1	2	3	4	0	1	2	3	4	0	1	2	3	4	0	1	2	3	4
gid						0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19
						2	1	3	2	1	7	1	2	1	4	2	1	2	1	1	2	1	2	1	1
Loop 1																									

Cuda - Problems/Optimization - Parallel Reduction

$$p_{ik} = \frac{e^{-\beta E_{ik}}}{\sum_{k'} \rho_{k'} e^{-\beta E_{ik'}}},$$

$$E_{ik} = \frac{(z_i - z_k)^2}{\sigma_i^2},$$

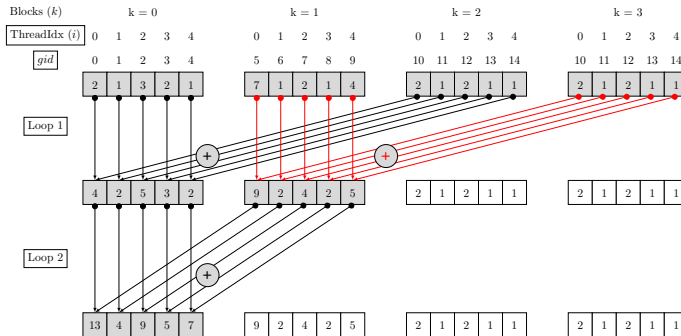
`off = blockSize * offset + tid` $\Rightarrow$ `sum[gid] += sum[off];`



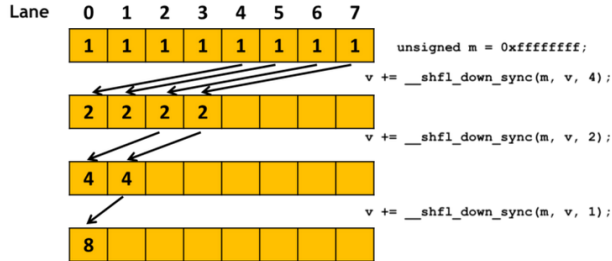
Cuda - Problems/Optimization - Parallel Reduction

$$p_{ik} = \frac{e^{-\beta E_{ik}}}{\sum_{k'} \rho_{k'} e^{-\beta E_{ik'}}}, \quad E_{ik} = \frac{(z_i - z_k)^2}{\sigma_i^2},$$

`off = blockSize * offset + tid` $\Rightarrow$ `sum[gid] += sum[off];`



$$z_k = \frac{\sum_i p_i p_{ik} z_i / \sigma_i^2}{\sum_i p_i p_{ik} / \sigma_i^2}, \quad (14)$$



Part of a warp-level parallel reduction using shfl_down_sync().

Figure: Ref: <https://developer.nvidia.com/blog/using-cuda-warp-level-primitives/>

```

608 z_0_numdem<<<1, N>>>(num_d, den_d, tracks_d, p_d, sig_d, N);

641 sumBlock_with_shfl_down<<<1, N>>>(num_d, num_d, N); // Ca
642 sumBlock_with_shfl_down<<<1, N>>>(den_d, den_d, N); // Ca

671 randDelta_h = CUDA_uniform_rand_list( MaxNvetex, randDelta_h);

701 kernel_T_0<<<1, N>>>(T_num_d, tracks_d, z_0, p_d, sig_d, N);
703 sumBlock_with_shfl_down<<<1, N>>>(T_num_d, T_num_d, N);

740 kernel_z_k_splitting<<<1, 1>>>(z_d, randDelta_d, CurrentNvetex);
746 kernel_z_k_splitting<<<1, 1>>>(z_d, randDelta_d, CurrentNvetex);

Do over beta_values

834 kernel_p_ik_num<<<CurrentNvetex, N>>>(p_ik_d, tracks_d, z_d, s
877 sumBlock_with_loop<<<half_vertex, N>>>(p_ik_den_d, p_ik_den_d,
895 kernel_p_ik<<<CurrentNvetex, N>>>(p_ik_d, p_ik_den_d, N, Curre

930 kernel_z_ik_num<<<CurrentNvetex, N>>>(p_ik_d, z_ik_num_d, p_d,
943 kernel_z_ik_den<<<CurrentNvetex, N>>>(p_ik_d, z_ik_den_d, p_d,
965 sumBlock_with_shfl_down_gid<<<CurrentNvetex, N>>>(z_ik_num_d,
985 sumBlock_with_shfl_down_gid<<<CurrentNvetex, N>>>(z_ik_den_d, z_ik_den_d, N);
1009 kernel_z_ik<<<CurrentNvetex, N>>>(z_d, z_ik_num_d, z_ik_den_d, N, CurrentNvetex);

```

$$z_0 = \frac{\sum_i p_i z_i / \sigma_i^2}{\sum_i p_i / \sigma_i^2}, \quad z_0 \pm \delta.$$

$$T_0 = 2 \frac{\sum_i \frac{p_i}{\sigma_i^2} \left(\frac{z_i - z_k}{\sigma_i} \right)^2}{\sum_i \frac{p_i}{\sigma_i^2}}.$$

$$p_{ik} = \frac{e^{-\beta E_{ik}}}{\sum_{k'} \rho_{k'} e^{-\beta E_{ik'}}}, \quad E_{ik} = (z_i - z_k)^2 / \sigma_i^2.$$

$$z_k = \frac{\sum_i p_i p_{ik} z_i / \sigma_i^2}{\sum_i p_i p_{ik} / \sigma_i^2}.$$

Figure: Kernels.

Kernels

```

608 z_0_numdem<<<1, N>>>(num_d, den_d, tracks_d, p_d, sig_d, N);
641 sumBlock_with_shfl_down<<<1, N>>>(num_d, num_d, N); // Ca
642 sumBlock_with_shfl_down<<<1, N>>>(den_d, den_d, N); // Ca

671 randDelta_h = CUDA_uniform_rand_list( MaxNvetex, randDelta_h);

701 kernel_T_0<<<1, N>>>(T_num_d, tracks_d, z_0, p_d, sig_d, N);
703 sumBlock_with_shfl_down<<<1, N>>>(T_num_d, T_num_d, N);

740 kernel_z_k_splitting<<<1, 1>>>(z_d, randDelta_d, CurrentNvetex);
746 kernel_z_k_splitting<<<1, 1>>>(z_d, randDelta_d, CurrentNvetex);

Do over beta_values

834 kernel_p_ik_num<<<CurrentNvetex, N>>>(p_ik_d, tracks_d, z_d, s
877 sumBlock_with_loop<<<half_vertex, N>>>(p_ik_den_d, p_ik_den_d,
895 kernel_p_ik<<<CurrentNvetex, N>>>(p_ik_d, p_ik_den_d, N, Curre

930 kernel_z_ik_num<<<CurrentNvetex, N>>>(p_ik_d, z_ik_num_d, p_d,
943 kernel_z_ik_den<<<CurrentNvetex, N>>>(p_ik_d, z_ik_den_d, p_d,
965 sumBlock_with_shfl_down_gid<<<CurrentNvetex, N>>>(z_ik_num_d,
985 sumBlock_with_shfl_down_gid<<<CurrentNvetex, N>>>(z_ik_den_d, z_ik_den_d, N);
1009 kernel_z_ik<<<CurrentNvetex, N>>>(z_d, z_ik_num_d, z_ik_den_d, N, CurrentNvetex);

```

$$z_0 = \frac{\sum_i p_i z_i / \sigma_i^2}{\sum_i p_i / \sigma_i^2}, \quad z_0 \pm \delta.$$

$$T_0 = 2 \frac{\sum_i \frac{p_i}{\sigma_i^2} \left(\frac{z_i - z_k}{\sigma_i} \right)^2}{\sum_i \frac{p_i}{\sigma_i^2}}.$$

$$p_{ik} = \frac{e^{-\beta E_{ik}}}{\sum_{k'} \rho_{k'} e^{-\beta E_{ik'}}}, \quad E_{ik} = (z_i - z_k)^2 / \sigma_i^2.$$

$$z_k = \frac{\sum_i p_i p_{ik} z_i / \sigma_i^2}{\sum_i p_i p_{ik} / \sigma_i^2}.$$

Figure: Kernels.

Kernels

```

608 z_0_numdem<<<1, N>>>(num_d, den_d, tracks_d, p_d, sig_d, N);
641 sumBlock_with_shfl_down <<<1, N>>>(num_d, num_d, N); // Ca
642 sumBlock_with_shfl_down <<<1, N>>>(den_d, den_d, N); // Ca

671 randDelta_h = CUDA_uniform_rand_list( MaxNvetex, randDelta_h);

701 kernel_T_0<<<1, N>>>(T_num_d, tracks_d, z_0, p_d, sig_d, N);
703 sumBlock_with_shfl_down <<<1, N>>>(T_num_d, T_num_d, N);

740 kernel_z_k_splitting <<<1, 1>>>(z_d, randDelta_d, CurrentNvetex);
746 kernel_z_k_splitting <<<1, 1>>>(z_d, randDelta_d, CurrentNvetex);

Do over beta_values

834 kernel_p_ik_num<<<CurrentNvetex, N>>>(p_ik_d, tracks_d, z_d, s
877 sumBlock_with_loop <<<half_vertex, N>>>(p_ik_den_d, p_ik_den_d,
895 kernel_p_ik<<<CurrentNvetex, N>>>(p_ik_d, p_ik_den_d, N, Curre

930 kernel_z_ik_num<<<CurrentNvetex, N>>>(p_ik_d, z_ik_num_d, p_d,
943 kernel_z_ik_den<<<CurrentNvetex, N>>>(p_ik_d, z_ik_den_d, p_d,
965 sumBlock_with_shfl_down_gid<<<CurrentNvetex, N>>>(z_ik_num_d,
985 sumBlock_with_shfl_down_gid<<<CurrentNvetex, N>>>(z_ik_den_d, z_ik_den_d, N);
1009 kernel_z_ik<<<CurrentNvetex, N>>>(z_d, z_ik_num_d, z_ik_den_d, N, CurrentNvetex);

```

$$z_0 = \frac{\sum_i p_i z_i / \sigma_i^2}{\sum_i p_i / \sigma_i^2}, \quad z_0 \pm \delta.$$

$$T_0 = 2 \frac{\sum_i \frac{p_i}{\sigma_i^2} \left(\frac{z_i - z_k}{\sigma_i} \right)^2}{\sum_i \frac{p_i}{\sigma_i^2}}.$$

$$p_{ik} = \frac{e^{-\beta E_{ik}}}{\sum_{k'} \rho_{k'} e^{-\beta E_{ik'}}}, \quad E_{ik} = (z_i - z_k)^2 / \sigma_i^2.$$

$$z_k = \frac{\sum_i p_i p_{ik} z_i / \sigma_i^2}{\sum_i p_i p_{ik} / \sigma_i^2}.$$

Figure: Kernels.

Kernels

```

608 z_0_numdem<<<1, N>>>(num_d, den_d, tracks_d, p_d, sig_d, N);
641 sumBlock_with_shfl_down<<<1, N>>>(num_d, num_d, N); // Ca
642 sumBlock_with_shfl_down<<<1, N>>>(den_d, den_d, N); // Ca
671 randDelta_h = CUDA_uniform_rand_list( MaxNvetex, randDelta_h);
701 kernel_T_0<<<1, N>>>(T_num_d, tracks_d, z_0, p_d, sig_d, N);
703 sumBlock_with_shfl_down<<<1, N>>>(T_num_d, T_num_d, N);
740 kernel_z_k_splitting<<<1, 1>>>(z_d, randDelta_d, CurrentNvetex);
746 kernel_z_k_splitting<<<1, 1>>>(z_d, randDelta_d, CurrentNvetex);

Do over beta_values

834 kernel_p_ik_num<<<CurrentNvetex, N>>>(p_ik_d, tracks_d, z_d, s
877 sumBlock_with_loop<<<half_vertex, N>>>(p_ik_den_d, p_ik_den_d,
895 kernel_p_ik<<<CurrentNvetex, N>>>(p_ik_d, p_ik_den_d, N, Curre

930 kernel_z_ik_num<<<CurrentNvetex, N>>>(p_ik_d, z_ik_num_d, p_d,
943 kernel_z_ik_den<<<CurrentNvetex, N>>>(p_ik_d, z_ik_den_d, p_d,
965 sumBlock_with_shfl_down_gid<<<CurrentNvetex, N>>>(z_ik_num_d,
985 sumBlock_with_shfl_down_gid<<<CurrentNvetex, N>>>(z_ik_den_d, z_ik_den_d, N);
1009 kernel_z_ik<<<CurrentNvetex, N>>>(z_d, z_ik_num_d, z_ik_den_d, N, CurrentNvetex);

```

$$z_0 = \frac{\sum_i p_i z_i / \sigma_i^2}{\sum_i p_i / \sigma_i^2}, \quad z_0 \pm \delta.$$

$$T_0 = 2 \frac{\sum_i \frac{p_i}{\sigma_i^2} \left(\frac{z_i - z_k}{\sigma_i} \right)^2}{\sum_i \frac{p_i}{\sigma_i^2}}.$$

$$p_{ik} = \frac{e^{-\beta E_{ik}}}{\sum_{k'} \rho_{k'} e^{-\beta E_{ik'}}}, \quad E_{ik} = (z_i - z_k)^2 / \sigma_i^2.$$

$$z_k = \frac{\sum_i p_i p_{ik} z_i / \sigma_i^2}{\sum_i p_i p_{ik} / \sigma_i^2}.$$

Figure: Kernels.

Kernels

```

608 z_0_numdem<<<1, N>>>(num_d, den_d, tracks_d, p_d, sig_d, N);

641 sumBlock_with_shfl_down<<<1, N>>>(num_d, num_d, N); // Ca
642 sumBlock_with_shfl_down<<<1, N>>>(den_d, den_d, N); // Ca

671 randDelta_h = CUDA_uniform_rand_list( MaxNvetex, randDelta_h);

701 kernel_T_0<<<1, N>>>(T_num_d, tracks_d, z_0, p_d, sig_d, N);
703 sumBlock_with_shfl_down<<<1, N>>>(T_num_d, T_num_d, N);

740 kernel_z_k_splitting<<<1, 1>>>(z_d, randDelta_d, CurrentNvetex);
746 kernel_z_k_splitting<<<1, 1>>>(z_d, randDelta_d, CurrentNvetex);

Do over beta_values

834 kernel_p_ik_num<<<CurrentNvetex, N>>>(p_ik_d, tracks_d, z_d, s
877 sumBlock_with_loop<<<half_vertex, N>>>(p_ik_den_d, p_ik_den_d,
895 kernel_p_ik<<<CurrentNvetex, N>>>(p_ik_d, p_ik_den_d, N, Curre

930 kernel_z_ik_num<<<CurrentNvetex, N>>>(p_ik_d, z_ik_num_d, p_d,
943 kernel_z_ik_den<<<CurrentNvetex, N>>>(p_ik_d, z_ik_den_d, p_d,
965 sumBlock_with_shfl_down_gid<<<CurrentNvetex, N>>>(z_ik_num_d,
985 sumBlock_with_shfl_down_gid<<<CurrentNvetex, N>>>(z_ik_den_d, z_ik_den_d, N);
1009 kernel_z_ik<<<CurrentNvetex, N>>>(z_d, z_ik_num_d, z_ik_den_d, N, CurrentNvetex);

```

$$z_0 = \frac{\sum_i p_i z_i / \sigma_i^2}{\sum_i p_i / \sigma_i^2}, \quad z_0 \pm \delta.$$

$$T_0 = 2 \frac{\sum_i \frac{p_i}{\sigma_i^2} \left(\frac{z_i - z_k}{\sigma_i} \right)^2}{\sum_i \frac{p_i}{\sigma_i^2}}.$$

$$p_{ik} = \frac{e^{-\beta E_{ik}}}{\sum_{k'} \rho_{k'} e^{-\beta E_{ik'}}}, \quad E_{ik} = (z_i - z_k)^2 / \sigma_i^2.$$

$$z_k = \frac{\sum_i p_i p_{ik} z_i / \sigma_i^2}{\sum_i p_i p_{ik} / \sigma_i^2}.$$

Figure: Kernels.

Kernels

```
608 z_0_numdem<<<1, N>>>(num_d, den_d, tracks_d, p_d, sig_d, N);
641 sumBlock_with_shfl_down<<<1, N>>>(num_d, num_d, N); // Ca
642 sumBlock_with_shfl_down<<<1, N>>>(den_d, den_d, N); // Ca

671 randDelta_h = CUDA_uniform_rand_list( MaxNvetex, randDelta_h);

701 kernel_T_0<<<1, N>>>(T_num_d, tracks_d, z_0, p_d, sig_d, N);
703 sumBlock_with_shfl_down<<<1, N>>>(T_num_d, T_num_d, N);

740 kernel_z_k_splitting<<<1, 1>>>(z_d, randDelta_d, CurrentNvetex);
746 kernel_z_k_splitting<<<1, 1>>>(z_d, randDelta_d, CurrentNvetex);

Do over beta_values

834 kernel_p_ik_num<<<CurrentNvetex, N>>>(p_ik_d, tracks_d, z_d, s
877 sumBlock_with_loop<<<half_vertex, N>>>(p_ik_den_d, p_ik_den_d,
895 kernel_p_ik<<<CurrentNvetex, N>>>(p_ik_d, p_ik_den_d, N, Curre

930 kernel_z_ik_num<<<CurrentNvetex, N>>>(p_ik_d, z_ik_num_d, p_d,
943 kernel_z_ik_den<<<CurrentNvetex, N>>>(p_ik_d, z_ik_den_d, p_d,
965 sumBlock_with_shfl_down_gid<<<CurrentNvetex, N>>>(z_ik_num_d,
985 sumBlock_with_shfl_down_gid<<<CurrentNvetex, N>>>(z_ik_den_d, z_ik_den_d, N);
1009 kernel_z_ik<<<CurrentNvetex, N>>>(z_d, z_ik_num_d, z_ik_den_d, N, CurrentNvetex);
```

$$z_0 = \frac{\sum_i p_i z_i / \sigma_i^2}{\sum_i p_i / \sigma_i^2}, \quad z_0 \pm \delta.$$

$$T_0 = 2 \frac{\sum_i \frac{p_i}{\sigma_i^2} \left(\frac{z_i - z_k}{\sigma_i} \right)^2}{\sum_i \frac{p_i}{\sigma_i^2}}.$$

$$p_{ik} = \frac{e^{-\beta E_{ik}}}{\sum_{k'} \rho_{k'} e^{-\beta E_{ik'}}}, \quad E_{ik} = (z_i - z_k)^2 / \sigma_i^2.$$

$$z_k = \frac{\sum_i p_i p_{ik} z_i / \sigma_i^2}{\sum_i p_i p_{ik} / \sigma_i^2}.$$

Figure: Kernels.

Kernels

```

608 z_0_numdem<<<1, N>>>(num_d, den_d, tracks_d, p_d, sig_d, N);

641 sumBlock_with_shfl_down<<<1, N>>>(num_d, num_d, N); // Ca
642 sumBlock_with_shfl_down<<<1, N>>>(den_d, den_d, N); // Ca

671 randDelta_h = CUDA_uniform_rand_list( MaxNvetex, randDelta_h);

701 kernel_T_0<<<1, N>>>(T_num_d, tracks_d, z_0, p_d, sig_d, N);
703 sumBlock_with_shfl_down<<<1, N>>>(T_num_d, T_num_d, N);

740 kernel_z_k_splitting<<<1, 1>>>(z_d, randDelta_d, CurrentNvetex);
746 kernel_z_k_splitting<<<1, 1>>>(z_d, randDelta_d, CurrentNvetex);

Do over beta_values

834 kernel_p_ik_num<<<CurrentNvetex, N>>>(p_ik_d, tracks_d, z_d, s
877 sumBlock_with_loop<<<half_vertex, N>>>(p_ik_den_d, p_ik_den_d,
895 kernel_p_ik<<<CurrentNvetex, N>>>(p_ik_d, p_ik_den_d, N, Curre

930 kernel_z_ik_num<<<CurrentNvetex, N>>>(p_ik_d, z_ik_num_d, p_d,
943 kernel_z_ik_den<<<CurrentNvetex, N>>>(p_ik_d, z_ik_den_d, p_d,
965 sumBlock_with_shfl_down_gid<<<CurrentNvetex, N>>>(z_ik_num_d,
985 sumBlock_with_shfl_down_gid<<<CurrentNvetex, N>>>(z_ik_den_d, z_ik_den_d, N);
1009 kernel_z_ik<<<CurrentNvetex, N>>>(z_d, z_ik_num_d, z_ik_den_d, N, CurrentNvetex);

```

$$z_0 = \frac{\sum_i p_i z_i / \sigma_i^2}{\sum_i p_i / \sigma_i^2}, \quad \underline{z_0 \pm \delta.}$$

$$T_0 = 2 \frac{\sum_i \frac{p_i}{\sigma_i^2} \left(\frac{z_i - z_k}{\sigma_i} \right)^2}{\sum_i \frac{p_i}{\sigma_i^2}}.$$

$$p_{ik} = \frac{e^{-\beta E_{ik}}}{\sum_{k'} \rho_{k'} e^{-\beta E_{ik'}}}, \quad E_{ik} = (z_i - z_k)^2 / \sigma_i^2.$$

$$z_k = \frac{\sum_i p_i p_{ik} z_i / \sigma_i^2}{\sum_i p_i p_{ik} / \sigma_i^2}.$$

Figure: Kernels.

Kernels

```

608 z_0_numdem<<<1, N>>>(num_d, den_d, tracks_d, p_d, sig_d, N);

641 sumBlock_with_shfl_down<<<1, N>>>(num_d, num_d, N); // Ca
642 sumBlock_with_shfl_down<<<1, N>>>(den_d, den_d, N); // Ca

671 randDelta_h = CUDA_uniform_rand_list( MaxNvetex, randDelta_h);

701 kernel_T_0<<<1, N>>>(T_num_d, tracks_d, z_0, p_d, sig_d, N);
703 sumBlock_with_shfl_down<<<1, N>>>(T_num_d, T_num_d, N);

740 kernel_z_k_splitting<<<1, 1>>>(z_d, randDelta_d, CurrentNvetex);
746 kernel_z_k_splitting<<<1, 1>>>(z_d, randDelta_d, CurrentNvetex);

Do over beta_values

834 kernel_p_ik_num<<<CurrentNvetex, N>>>(p_ik_d, tracks_d, z_d, s
877 sumBlock_with_loop<<<half-vertex, N>>>(p_ik_den_d, p_ik_den_d,
895 kernel_p_ik<<<CurrentNvetex, N>>>(p_ik_d, p_ik_den_d, N, Curre

930 kernel_z_ik_num<<<CurrentNvetex, N>>>(p_ik_d, z_ik_num_d, p_d,
943 kernel_z_ik_den<<<CurrentNvetex, N>>>(p_ik_d, z_ik_den_d, p_d,
965 sumBlock_with_shfl_down_gid<<<CurrentNvetex, N>>>(z_ik_num_d,
985 sumBlock_with_shfl_down_gid<<<CurrentNvetex, N>>>(z_ik_den_d, z_ik_den_d, N);
1009 kernel_z_ik<<<CurrentNvetex, N>>>(z_d, z_ik_num_d, z_ik_den_d, N, CurrentNvetex);

```

$$z_0 = \frac{\sum_i p_i z_i / \sigma_i^2}{\sum_i p_i / \sigma_i^2}, \quad z_0 \pm \delta.$$

$$T_0 = 2 \frac{\sum_i \frac{p_i}{\sigma_i^2} \left(\frac{z_i - z_k}{\sigma_i} \right)^2}{\sum_i \frac{p_i}{\sigma_i^2}}.$$

$$p_{ik} = \frac{e^{-\beta E_{ik}}}{\sum_{k'} \rho_{k'} e^{-\beta E_{ik'}}}, \quad E_{ik} = (z_i - z_k)^2 / \sigma_i^2.$$

$$z_k = \frac{\sum_i p_i p_{ik} z_i / \sigma_i^2}{\sum_i p_i p_{ik} / \sigma_i^2}.$$

Figure: Kernels.

Kernels

```

608 z_0_numdem<<<1, N>>>(num_d, den_d, tracks_d, p_d, sig_d, N);

641 sumBlock_with_shfl_down<<<1, N>>>(num_d, num_d, N); // Ca
642 sumBlock_with_shfl_down<<<1, N>>>(den_d, den_d, N); // Ca

671 randDelta_h = CUDA_uniform_rand_list( MaxNvetex, randDelta_h);

701 kernel_T_0<<<1, N>>>(T_num_d, tracks_d, z_0, p_d, sig_d, N);
703 sumBlock_with_shfl_down<<<1, N>>>(T_num_d, T_num_d, N);

740 kernel_z_k_splitting<<<1, 1>>>(z_d, randDelta_d, CurrentNvetex);
746 kernel_z_k_splitting<<<1, 1>>>(z_d, randDelta_d, CurrentNvetex);

Do over beta_values

834 kernel_p_ik_num<<<CurrentNvetex, N>>>(p_ik_d, tracks_d, z_d, s
877 sumBlock_with_loop <<<half_vertex, N>>>(p_ik_den_d, p_ik_den_d,
895 kernel_p_ik<<<CurrentNvetex, N>>>(p_ik_d, p_ik_den_d, N, Curre

930 kernel_z_ik_num<<<CurrentNvetex, N>>>(p_ik_d, z_ik_num_d, p_d,
943 kernel_z_ik_den<<<CurrentNvetex, N>>>(p_ik_d, z_ik_den_d, p_d,
965 sumBlock_with_shfl_down_gid<<<CurrentNvetex, N>>>(z_ik_num_d,
985 sumBlock_with_shfl_down_gid<<<CurrentNvetex, N>>>(z_ik_den_d, z_ik_den_d, N);
1009 kernel_z_ik<<<CurrentNvetex, N>>>(z_d, z_ik_num_d, z_ik_den_d, N, CurrentNvetex);

```

$$z_0 = \frac{\sum_i p_i z_i / \sigma_i^2}{\sum_i p_i / \sigma_i^2}, \quad z_0 \pm \delta.$$

$$T_0 = 2 \frac{\sum_i \frac{p_i}{\sigma_i^2} \left(\frac{z_i - z_k}{\sigma_i} \right)^2}{\sum_i \frac{p_i}{\sigma_i^2}}.$$

$$p_{ik} = \frac{e^{-\beta E_{ik}}}{\sum_{k'} \rho_{k'} e^{-\beta E_{ik'}}}, \quad E_{ik} = (z_i - z_k)^2 / \sigma_i^2.$$

$$z_k = \frac{\sum_i p_i p_{ik} z_i / \sigma_i^2}{\sum_i p_i p_{ik} / \sigma_i^2}.$$

Figure: Kernels.

```

608 z_0_numdem<<<1, N>>>(num_d, den_d, tracks_d, p_d, sig_d, N);

641 sumBlock_with_shfl_down<<<1, N>>>(num_d, num_d, N); // Ca
642 sumBlock_with_shfl_down<<<1, N>>>(den_d, den_d, N); // Ca

671 randDelta_h = CUDA_uniform_rand_list( MaxNvetex, randDelta_h);

701 kernel_T_0<<<1, N>>>(T_num_d, tracks_d, z_0, p_d, sig_d, N);
703 sumBlock_with_shfl_down<<<1, N>>>(T_num_d, T_num_d, N);

740 kernel_z_k_splitting<<<1, 1>>>(z_d, randDelta_d, CurrentNvetex);
746 kernel_z_k_splitting<<<1, 1>>>(z_d, randDelta_d, CurrentNvetex);

Do over beta_values

834 kernel_p_ik_num<<<CurrentNvetex, N>>>(p_ik_d, tracks_d, z_d, s
877 sumBlock_with_loop<<<half_vertex, N>>>(p_ik_den_d, p_ik_den_d,
895 kernel_p_ik<<<CurrentNvetex, N>>>(p_ik_d, p_ik_den_d, N, Curre

930 kernel_z_ik_num<<<CurrentNvetex, N>>>(p_ik_d, z_ik_num_d, p_d,
943 kernel_z_ik_den<<<CurrentNvetex, N>>>(p_ik_d, z_ik_den_d, p_d,
965 sumBlock_with_shfl_down_gid<<<CurrentNvetex, N>>>(z_ik_num_d,
985 sumBlock_with_shfl_down_gid<<<CurrentNvetex, N>>>(z_ik_den_d, z_ik_den_d, N);
1009 kernel_z_ik<<<CurrentNvetex, N>>>(z_d, z_ik_num_d, z_ik_den_d, N, CurrentNvetex);

```

$$z_0 = \frac{\sum_i p_i z_i / \sigma_i^2}{\sum_i p_i / \sigma_i^2}, \quad z_0 \pm \delta.$$

$$T_0 = 2 \frac{\sum_i \frac{p_i}{\sigma_i^2} \left(\frac{z_i - z_k}{\sigma_i} \right)^2}{\sum_i \frac{p_i}{\sigma_i^2}}.$$

$$p_{ik} = \frac{e^{-\beta E_{ik}}}{\sum_{k'} \rho_{k'} e^{-\beta E_{ik'}}}, \quad E_{ik} = (z_i - z_k)^2 / \sigma_i^2.$$

$$z_k = \frac{\sum_i p_i p_{ik} z_i / \sigma_i^2}{\sum_i p_i p_{ik} / \sigma_i^2}.$$

Figure: Kernels.

Kernels

```

608 z_0_numdem<<<1, N>>>(num_d, den_d, tracks_d, p_d, sig_d, N);

641 sumBlock_with_shfl_down<<<1, N>>>(num_d, num_d, N); // Ca
642 sumBlock_with_shfl_down<<<1, N>>>(den_d, den_d, N); // Ca

671 randDelta_h = CUDA_uniform_rand_list( MaxNvetex, randDelta_h);

701 kernel_T_0<<<1, N>>>(T_num_d, tracks_d, z_0, p_d, sig_d, N);
703 sumBlock_with_shfl_down<<<1, N>>>(T_num_d, T_num_d, N);

740 kernel_z_k_splitting<<<1, 1>>>(z_d, randDelta_d, CurrentNvetex);
746 kernel_z_k_splitting<<<1, 1>>>(z_d, randDelta_d, CurrentNvetex);

Do over beta_values

834 kernel_p_ik_num<<<CurrentNvetex, N>>>(p_ik_d, tracks_d, z_d, s
877 sumBlock_with_loop<<<half_vertex, N>>>(p_ik_den_d, p_ik_den_d,
895 kernel_p_ik<<<CurrentNvetex, N>>>(p_ik_d, p_ik_den_d, N, Curre

930 kernel_z_ik_num<<<CurrentNvetex, N>>>(p_ik_d, z_ik_num_d, p_d,
943 kernel_z_ik_den<<<CurrentNvetex, N>>>(p_ik_d, z_ik_den_d, p_d,
965 sumBlock_with_shfl_down_gid<<<CurrentNvetex, N>>>(z_ik_num_d,
985 sumBlock_with_shfl_down_gid<<<CurrentNvetex, N>>>(z_ik_den_d, z_ik_den_d, N);
1009 kernel_z_ik<<<CurrentNvetex, N>>>(z_d, z_ik_num_d, z_ik_den_d, N, CurrentNvetex);

```

$$z_0 = \frac{\sum_i p_i z_i / \sigma_i^2}{\sum_i p_i / \sigma_i^2}, \quad z_0 \pm \delta.$$

$$T_0 = 2 \frac{\sum_i \frac{p_i}{\sigma_i^2} \left(\frac{z_i - z_k}{\sigma_i} \right)^2}{\sum_i \frac{p_i}{\sigma_i^2}}.$$

$$p_{ik} = \frac{e^{-\beta E_{ik}}}{\sum_{k'} \rho_{k'} e^{-\beta E_{ik'}}}, \quad E_{ik} = (z_i - z_k)^2 / \sigma_i^2.$$

$$z_k = \frac{\sum_i p_i p_{ik} z_i / \sigma_i^2}{\sum_i p_i p_{ik} / \sigma_i^2}.$$

Figure: Kernels.

Kernels

```

608 z_0_numdem<<<1, N>>>(num_d, den_d, tracks_d, p_d, sig_d, N);

641 sumBlock_with_shfl_down<<<1, N>>>(num_d, num_d, N); // Ca
642 sumBlock_with_shfl_down<<<1, N>>>(den_d, den_d, N); // Ca

671 randDelta_h = CUDA_uniform_rand_list( MaxNvetex, randDelta_h);

701 kernel_T_0<<<1, N>>>(T_num_d, tracks_d, z_0, p_d, sig_d, N);
703 sumBlock_with_shfl_down<<<1, N>>>(T_num_d, T_num_d, N);

740 kernel_z_k_splitting<<<1, 1>>>(z_d, randDelta_d, CurrentNvetex);
746 kernel_z_k_splitting<<<1, 1>>>(z_d, randDelta_d, CurrentNvetex);

Do over beta_values

834 kernel_p_ik_num<<<CurrentNvetex, N>>>(p_ik_d, tracks_d, z_d, s
877 sumBlock_with_loop<<<half_vertex, N>>>(p_ik_den_d, p_ik_den_d,
895 kernel_p_ik<<<CurrentNvetex, N>>>(p_ik_d, p_ik_den_d, N, Curre

930 kernel_z_ik_num<<<CurrentNvetex, N>>>(p_ik_d, z_ik_num_d, p_d,
943 kernel_z_ik_den<<<CurrentNvetex, N>>>(p_ik_d, z_ik_den_d, p_d,
965 sumBlock_with_shfl_down_gid<<<CurrentNvetex, N>>>(z_ik_num_d,
985 sumBlock_with_shfl_down_gid<<<CurrentNvetex, N>>>(z_ik_den_d, z_ik_den_d, N);
1009 kernel_z_ik<<<CurrentNvetex, N>>>(z_d, z_ik_num_d, z_ik_den_d, N, CurrentNvetex);

```

$$z_0 = \frac{\sum_i p_i z_i / \sigma_i^2}{\sum_i p_i / \sigma_i^2}, \quad z_0 \pm \delta.$$

$$T_0 = 2 \frac{\sum_i \frac{p_i}{\sigma_i^2} \left(\frac{z_i - z_k}{\sigma_i} \right)^2}{\sum_i \frac{p_i}{\sigma_i^2}}.$$

$$p_{ik} = \frac{e^{-\beta E_{ik}}}{\sum_{k'} \rho_{k'} e^{-\beta E_{ik'}}}, \quad E_{ik} = (z_i - z_k)^2 / \sigma_i^2.$$

$$z_k = \frac{\sum_i p_i p_{ik} z_i / \sigma_i^2}{\sum_i p_i p_{ik} / \sigma_i^2}.$$

Figure: Kernels.

```

608 z_0_numdem<<<1, N>>>(num_d, den_d, tracks_d, p_d, sig_d, N);

641 sumBlock_with_shfl_down<<<1, N>>>(num_d, num_d, N); // Ca
642 sumBlock_with_shfl_down<<<1, N>>>(den_d, den_d, N); // Ca

671 randDelta_h = CUDA_uniform_rand_list( MaxNvetex, randDelta_h);

701 kernel_T_0<<<1, N>>>(T_num_d, tracks_d, z_0, p_d, sig_d, N);
703 sumBlock_with_shfl_down<<<1, N>>>(T_num_d, T_num_d, N);

740 kernel_z_k_splitting<<<1, 1>>>(z_d, randDelta_d, CurrentNvetex);
746 kernel_z_k_splitting<<<1, 1>>>(z_d, randDelta_d, CurrentNvetex);

Do over beta_values

834 kernel_p_ik_num<<<CurrentNvetex, N>>>(p_ik_d, tracks_d, z_d, s
877 sumBlock_with_loop<<<half_vertex, N>>>(p_ik_den_d, p_ik_den_d,
895 kernel_p_ik<<<CurrentNvetex, N>>>(p_ik_d, p_ik_den_d, N, Curre

930 kernel_z_ik_num<<<CurrentNvetex, N>>>(p_ik_d, z_ik_num_d, p_d,
943 kernel_z_ik_den<<<CurrentNvetex, N>>>(p_ik_d, z_ik_den_d, p_d,
965 sumBlock_with_shfl_down_gid<<<CurrentNvetex, N>>>(z_ik_num_d,
985 sumBlock_with_shfl_down_gid<<<CurrentNvetex, N>>>(z_ik_den_d, z_ik_den_d, N);
1009 kernel_z_ik<<<CurrentNvetex, N>>>(z_d, z_ik_num_d, z_ik_den_d, N, CurrentNvetex);
    
```

$$z_0 = \frac{\sum_i p_i z_i / \sigma_i^2}{\sum_i p_i / \sigma_i^2}, \quad z_0 \pm \delta.$$

$$T_0 = 2 \frac{\sum_i \frac{p_i}{\sigma_i^2} \left(\frac{z_i - z_k}{\sigma_i} \right)^2}{\sum_i \frac{p_i}{\sigma_i^2}}.$$

$$p_{ik} = \frac{e^{-\beta E_{ik}}}{\sum_{k'} \rho_{k'} e^{-\beta E_{ik'}}}, \quad E_{ik} = (z_i - z_k)^2 / \sigma_i^2.$$

$$z_k = \frac{\sum_i p_i p_{ik} z_i / \sigma_i^2}{\sum_i p_i p_{ik} / \sigma_i^2}.$$

Figure: Kernels.


```

608 z_0_numdem<<<1, N>>>(num_d, den_d, tracks_d, p_d, sig_d, N);

641 sumBlock_with_shfl_down<<<1, N>>>(num_d, num_d, N); // Ca
642 sumBlock_with_shfl_down<<<1, N>>>(den_d, den_d, N); // Ca

671 randDelta_h = CUDA_uniform_rand_list( MaxNvetex, randDelta_h);

701 kernel_T_0<<<1, N>>>(T_num_d, tracks_d, z_0, p_d, sig_d, N);
703 sumBlock_with_shfl_down<<<1, N>>>(T_num_d, T_num_d, N);

740 kernel_z_k_splitting<<<1, 1>>>(z_d, randDelta_d, CurrentNvetex);
746 kernel_z_k_splitting<<<1, 1>>>(z_d, randDelta_d, CurrentNvetex);

Do over beta_values

834 kernel_p_ik_num<<<CurrentNvetex, N>>>(p_ik_d, tracks_d, z_d, s
877 sumBlock_with_loop<<<half_vertex, N>>>(p_ik_den_d, p_ik_den_d,
895 kernel_p_ik<<<CurrentNvetex, N>>>(p_ik_d, p_ik_den_d, N, Curre

930 kernel_z_ik_num<<<CurrentNvetex, N>>>(p_ik_d, z_ik_num_d, p_d,
943 kernel_z_ik_den<<<CurrentNvetex, N>>>(p_ik_d, z_ik_den_d, p_d,
965 sumBlock_with_shfl_down_gid<<<CurrentNvetex, N>>>(z_ik_num_d,
985 sumBlock_with_shfl_down_gid<<<CurrentNvetex, N>>>(z_ik_den_d, z_ik_den_d, N);
1009 kernel_z_ik<<<CurrentNvetex, N>>>(z_d, z_ik_num_d, z_ik_den_d, N, CurrentNvetex);

```

$$z_0 = \frac{\sum_i p_i z_i / \sigma_i^2}{\sum_i p_i / \sigma_i^2}, \quad z_0 \pm \delta.$$

$$T_0 = 2 \frac{\sum_i \frac{p_i}{\sigma_i^2} \left(\frac{z_i - z_k}{\sigma_i} \right)^2}{\sum_i \frac{p_i}{\sigma_i^2}}.$$

$$p_{ik} = \frac{e^{-\beta E_{ik}}}{\sum_{k'} \rho_{k'} e^{-\beta E_{ik'}}}, \quad E_{ik} = (z_i - z_k)^2 / \sigma_i^2.$$

$$z_k = \frac{\sum_i p_i p_{ik} z_i / \sigma_i^2}{\sum_i p_i p_{ik} / \sigma_i^2}.$$

Figure: Kernels.

```

608 z_0_numdem<<<1, N>>>(num_d, den_d, tracks_d, p_d, sig_d, N);

641 sumBlock_with_shfl_down<<<1, N>>>(num_d, num_d, N); // Ca
642 sumBlock_with_shfl_down<<<1, N>>>(den_d, den_d, N); // Ca

671 randDelta_h = CUDA_uniform_rand_list( MaxNvetex, randDelta_h);

701 kernel_T_0<<<1, N>>>(T_num_d, tracks_d, z_0, p_d, sig_d, N);
703 sumBlock_with_shfl_down<<<1, N>>>(T_num_d, T_num_d, N);

740 kernel_z_k_splitting<<<1, 1>>>(z_d, randDelta_d, CurrentNvetex);
746 kernel_z_k_splitting<<<1, 1>>>(z_d, randDelta_d, CurrentNvetex);

Do over beta_values

834 kernel_p_ik_num<<<CurrentNvetex, N>>>(p_ik_d, tracks_d, z_d, s
877 sumBlock_with_loop<<<half_vertex, N>>>(p_ik_den_d, p_ik_den_d,
895 kernel_p_ik<<<CurrentNvetex, N>>>(p_ik_d, p_ik_den_d, N, Curre

930 kernel_z_ik_num<<<CurrentNvetex, N>>>(p_ik_d, z_ik_num_d, p_d,
943 kernel_z_ik_den<<<CurrentNvetex, N>>>(p_ik_d, z_ik_den_d, p_d,
965 sumBlock_with_shfl_down_gid<<<CurrentNvetex, N>>>(z_ik_num_d,
985 sumBlock_with_shfl_down_gid<<<CurrentNvetex, N>>>(z_ik_den_d, z_ik_den_d, N);
1009 kernel_z_ik<<<CurrentNvetex, N>>>(z_d, z_ik_num_d, z_ik_den_d, N, CurrentNvetex);

```

$$z_0 = \frac{\sum_i p_i z_i / \sigma_i^2}{\sum_i p_i / \sigma_i^2}, \quad z_0 \pm \delta.$$

$$T_0 = 2 \frac{\sum_i \frac{p_i}{\sigma_i^2} \left(\frac{z_i - z_k}{\sigma_i} \right)^2}{\sum_i \frac{p_i}{\sigma_i^2}}.$$

$$p_{ik} = \frac{e^{-\beta E_{ik}}}{\sum_{k'} \rho_{k'} e^{-\beta E_{ik'}}}, \quad E_{ik} = (z_i - z_k)^2 / \sigma_i^2.$$

$$z_k = \frac{\sum_i p_i p_{ik} z_i / \sigma_i^2}{\sum_i p_i p_{ik} / \sigma_i^2}.$$

Figure: Kernels.

```

608 z_0_numdem<<<1, N>>>(num_d, den_d, tracks_d, p_d, sig_d, N);

641 sumBlock_with_shfl_down<<<1, N>>>(num_d, num_d, N); // Ca
642 sumBlock_with_shfl_down<<<1, N>>>(den_d, den_d, N); // Ca

671 randDelta_h = CUDA_uniform_rand_list( MaxNvetex, randDelta_h);

701 kernel_T_0<<<1, N>>>(T_num_d, tracks_d, z_0, p_d, sig_d, N);
703 sumBlock_with_shfl_down<<<1, N>>>(T_num_d, T_num_d, N);

740 kernel_z_k_splitting<<<1, 1>>>(z_d, randDelta_d, CurrentNvetex);
746 kernel_z_k_splitting<<<1, 1>>>(z_d, randDelta_d, CurrentNvetex);

Do over beta_values

834 kernel_p_ik_num<<<CurrentNvetex, N>>>(p_ik_d, tracks_d, z_d, s
877 sumBlock_with_loop<<<half_vertex, N>>>(p_ik_den_d, p_ik_den_d,
895 kernel_p_ik<<<CurrentNvetex, N>>>(p_ik_d, p_ik_den_d, N, Curre

930 kernel_z_ik_num<<<CurrentNvetex, N>>>(p_ik_d, z_ik_num_d, p_d,
943 kernel_z_ik_den<<<CurrentNvetex, N>>>(p_ik_d, z_ik_den_d, p_d,
965 sumBlock_with_shfl_down_gid<<<CurrentNvetex, N>>>(z_ik_num_d,
985 sumBlock_with_shfl_down_gid<<<CurrentNvetex, N>>>(z_ik_den_d, z_ik_den_d, N);
1009 kernel_z_ik<<<CurrentNvetex, N>>>(z_d, z_ik_num_d, z_ik_den_d, N, CurrentNvetex);

```

$$z_0 = \frac{\sum_i p_i z_i / \sigma_i^2}{\sum_i p_i / \sigma_i^2}, \quad z_0 \pm \delta.$$

$$T_0 = 2 \frac{\sum_i \frac{p_i}{\sigma_i^2} \left(\frac{z_i - z_k}{\sigma_i} \right)^2}{\sum_i \frac{p_i}{\sigma_i^2}}.$$

$$p_{ik} = \frac{e^{-\beta E_{ik}}}{\sum_{k'} \rho_{k'} e^{-\beta E_{ik'}}}, \quad E_{ik} = (z_i - z_k)^2 / \sigma_i^2.$$

$$z_k = \frac{\sum_i p_i p_{ik} z_i / \sigma_i^2}{\sum_i p_i p_{ik} / \sigma_i^2}.$$

Figure: Kernels.

Results with CUDA Implementation

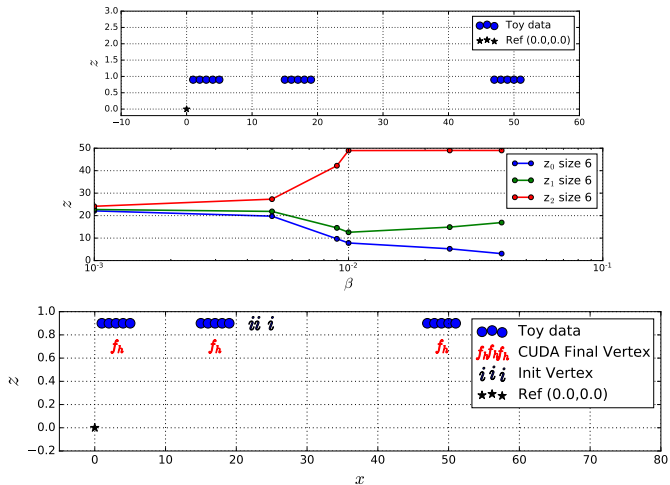


Figure: Toy data (on top). Middle figure corresponds to the vertex position evolution through annealing step. Bottom figure shows the data and the final vertex position after six annealing steps.

Conclusions

- ☐ A first CUDA version for DA ;
- ☐ Extrapolate for even and odd number of vertex;
- ☐ Optimize memory transfers;
- ☐ Streams;

Conclusions

- A first CUDA version for *DA*;
- Extrapolate for even and odd number of vertex;
- Optimize memory transfers;
- Streams;

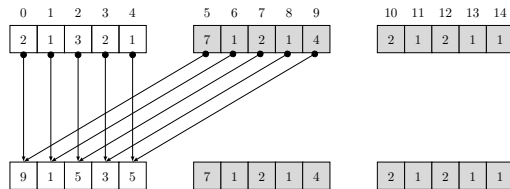


Figure: Parallel reduction for odd n . vertex.

Conclusions

- Extrapolate for even and odd number of vertex;
- Optimize memory transfers;
- Streams;

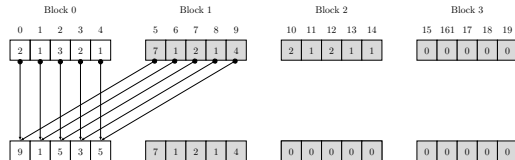


Figure: Parallel reduction for odd n . vertex.
Possibility round up to archive 2^n .

Conclusions

- Extrapolate for even and odd number of vertex;
- Optimize memory transfers;
- Streams;

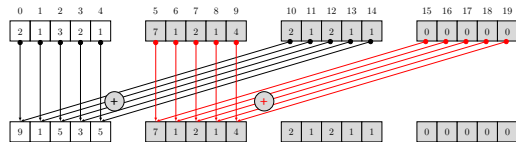


Figure: Parallel reduction for odd n . vertex.
Possibility round up to archive 2^n .

Conclusions

- Extrapolate for even and odd number of vertex;
- Optimize memory transfers;
- Streams;

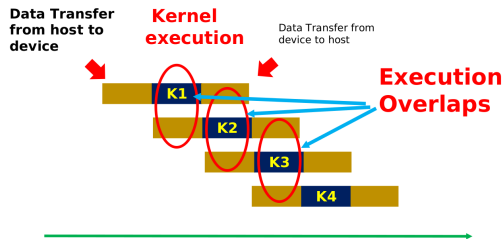


Figure: Cuda programming Masterclass.
Kasun Liyanage.

- Avoid `cudaMalloc*()`, `cudaHostAlloc()`, `cudaFree*()`, `cudaHostRegister()`, `cudaHostUnregister()` on every event ;
- Use `cudaMemcpyAsync()`, `cudaMemsetAsync()`, `cudaMemPrefetchAsync()` etc.
- Synchronization needs should be fulfilled with `ExternalWork` extension to `EDProducers`;
- Within `acquire()/produce()`, the current CUDA device is set implicitly and the CUDA stream is provided by the system (with `cms::cuda::ScopedContextAcquire/ cms::cuda::ScopedContextProduce`)...

The End

Cuda - Problems/Optimization - Memory Access - Streams.

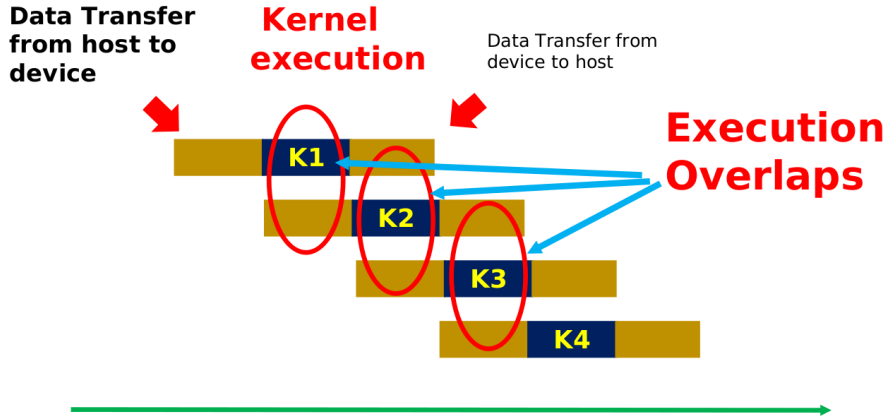


Figure: Cuda programming Masterclass. Kasun Liyanage.

□ <https://github.com/cms-patatrack>



The Patatrack group



- Patatrack was formed by people with common interest and a varied pool of expertise
 - Software optimisation
 - Heterogeneous architectures
 - Track reconstruction
 - High Level Trigger
- Work started in 2016 with the participation to the EuroHack 2016 event, sponsored by NVIDIA
- And continued through 2017 to 2019 with self-organized Hackathons at CERN, collaboration with Openlab, training and working with students, and so on

- Categorizing CUDA applications
 - Bandwidth bound applications;
 - Computation bound applications (e.g. if, else statements).

- Warp divergence;

```
...  
if ( threadIdx.x < 16 )  
{  
    ... A ...  
}  
else  
{  
    ... B ...  
}  
...
```

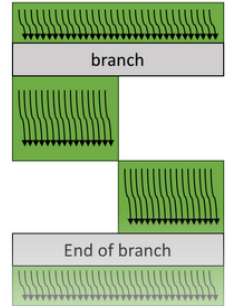


Figure: Learn-CUDA-Programming. Bharatkumar Sharma, Jaegeun Han (2019).

Cuda - Problems/Optimization - Warp divergence

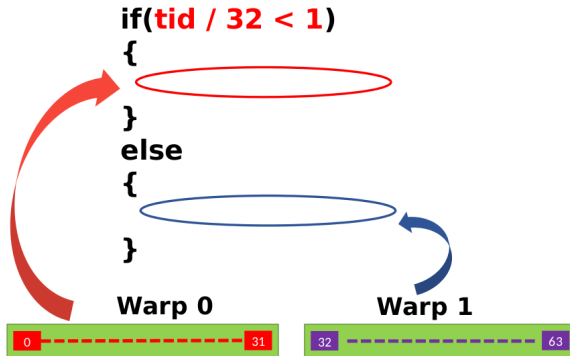


Figure: No warp divergence.

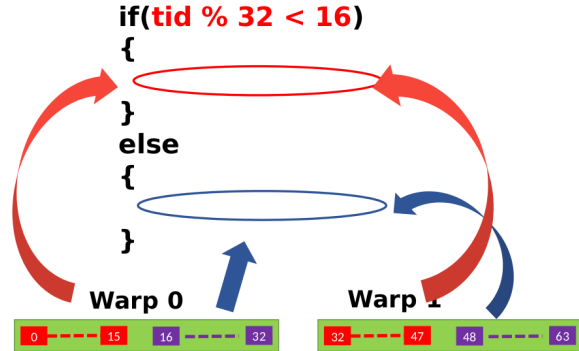


Figure: Warp divergence. Cuda programming Masterclass.

CUDA Implementation

□ Design goals:

- Provide a mechanism for a chain of modules to share a resource
- Resource can be e.g. CUDA device memory or a CUDA stream
- Minimize data movements between the CPU and the device
- Support multiple devices
- Allow the same job configuration to be used on all hardware combinations

Ref.: <https://github.com/cms-patatrack/cmssw/blob/master/HeterogeneousCore/CUDACore/README.md>

□ Overall guidelines:

- Avoid `cudaMalloc*`(), `cudaHostAlloc()`, `cudaFree*`(), `cudaHostRegister()`, `cudaHostUnregister()` on every event
- Use `cudaMemcpyAsync()`, `cudaMemsetAsync()`, `cudaMemPrefetchAsync()` etc.
- Synchronization needs should be fulfilled with `ExternalWork` extension to `EDProducers`
- Within `acquire()/produce()`, the current CUDA device is set implicitly and the CUDA stream is provided by the system (with `cms::cuda::ScopedContextAcquire/` `cms::cuda::ScopedContextProduce`)

DA: CUDA/C++ Implementation

□ DA CUDA KERNEL STARTED

```
__global__ void CalcExponentDenom(float* Exp_ik, float* z_i, float* z_k, float* dz2, float beta, int dataSize, int
numberOfvertex) {
    // This kernel calculates parallel the exponent of p_ik
    // Exp_ik Exponent: Energy multiplied by -beta
    // c is the result
    // z_i tracks // z-coordinate at point of closest approach to the beamline
    // z_k vertex
    // dz2 square of the error

    int tid = threadIdx.x; // thread Id
    int bid = blockIdx.x; // block Id
    int gid = blockIdx.x * blockDim.x + tid;
    int TotalSize = dataSize*numberOfvertex; // nTracks * nVertex

    if (gid < TotalSize) {
        Exp_ik[gid] = expf(-beta*(z_i[tid] - z_k[bid])*(z_i[tid] - z_k[bid]));
    }
}
```

DA: CUDA/C++ Implementation

□ DA CUDA KERNELs STARTED

```
0 kernel5<<<1, N>>>(num_d, den_d, tracks_d, p_d, sig_d, N);
1 sumBlock_with_shfl_down<<<1, N>>>(num_d, num_d, N); // Calculating numerator sum for z_0
2 sumBlock_with_shfl_down<<<1, N>>>(den_d, den_d, N); // Calculating denominator sum for z_0
3 randDelta_h = CUDA_uniform_rand_list( MaxNvetex, randDelta_h); // Do not forget to free(numbers_h);
4 kernel_T_0<<<1, N>>>(T_num_d, tracks_d, z_0, p_d, sig_d, N); // Calculating numerator for T_0
5 sumBlock_with_shfl_down<<<1, N>>>(T_num_d, T_num_d, N); // Calculating numerator sum for T_0
6 kernel_z_k_splitting<<<1, 1>>>(z_d, randDelta_d, CurrentNvetex);
7 kernel_z_k_splitting<<<1, 1>>>(z_d, randDelta_d, CurrentNvetex);
8 kernel_p_ik_num<<<CurrentNvetex, N>>>(p_ik_d, tracks_d, z_d, sig_d, beta, N, CurrentNvetex);
9 kernel_p_ik_den<<<3, N>>>(p_ik_d, p_ik_den_d, rho_d, N);
10 kernel_p_ik<<<CurrentNvetex, N>>>(p_ik_d, p_ik_den_d, N, CurrentNvetex);
11 kernel_z_ik_num<<<CurrentNvetex, N>>>(p_ik_d, z_ik_num_d, p_d, tracks_d, sig_d, N, CurrentNvetex);
12 kernel_z_ik_den<<<CurrentNvetex, N>>>(p_ik_d, z_ik_den_d, p_d, tracks_d, sig_d, N, CurrentNvetex);
13 sumBlock_with_shfl_down_gid<<<CurrentNvetex, N>>>(z_ik_num_d, z_ik_num_d, N);
14 sumBlock_with_shfl_down_gid<<<CurrentNvetex, N>>>(z_ik_den_d, z_ik_den_d, N);
15 kernel_z_ik<<<CurrentNvetex, N>>>(z_d, z_ik_num_d, z_ik_den_d, N, CurrentNvetex);
```

□ Compile with nvprof. nvprof Profile modes:

- Summary mode
- GPU and API trace mode
 - kernel executions.
 - memory transfers.
- Event metrics summary mode
- Event, metrics trace mode

□ Some nvprof metrics

- `Achieved_occupancy`
- `Branch_efficiency`
- `stall_memory_throttle`
- etc

CMSSW: CUDA Track data

cmssw/CUDADataFormats/Track/interface/PixelTrackHeterogeneous.h

cms-patatrack / **cmssw**

forked from cms-sw/cmssw

Watch 12



<> Code

Issues 48

Pull requests 15

Actions

Projects

Wiki

Security

Insights

master

cmssw / CUDADataFormats / Track / interface / PixelTrackHeterogeneous.h

Go to file



VinInn Move hit indexes to 32 bits (#583)

Latest commit 83081a8 17 days ago

History

2 contributors



74 lines (56 sloc) 2.52 KB

Raw

Blame



```
1 #ifndef CUDADataFormatsTrackTrackHeterogeneous_H
2 #define CUDADataFormatsTrackTrackHeterogeneous_H
3
4 #include "CUDADataFormats/Track/interface/TrajectoryStateSoA.h"
5 #include "HeterogeneousCore/CUDAUtilities/interface/HistoContainer.h"
6
7 #include "CUDADataFormats/Common/interface/HeterogeneousSoA.h"
8
9 namespace trackQuality {
10     enum Quality : uint8_t { bad = 0, dup, loose, strict, tight, highPurity };
11 }
12
13 template <int32_t S>
14 class TrackSoAT {
15 public:
16     static constexpr int32_t stride() { return S; }
```